
Practical Session 1: Mutex Lock Demonstration

File: practical1_mutex.c

```
``c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 10

typedef struct {
    int tid;
} thread_data_t;

int shared_counter = 0;
pthread_mutex_t count_mutex;

void* thr_func(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;
    // Lock the mutex before accessing the shared counter
    pthread_mutex_lock(&count_mutex);
    for (int i = 0; i < 100000; i++) {
        shared_counter++;
    }
    pthread_mutex_unlock(&count_mutex);
    pthread_exit(NULL);
}

int main() {
    pthread_t thr[NUM_THREADS];
    thread_data_t thr_data[NUM_THREADS];
    int i, rc;

    shared_counter = 0;
    pthread_mutex_init(&count_mutex, NULL);

    for (i = 0; i < NUM_THREADS; ++i) {
        thr_data[i].tid = i;
        if ((rc = pthread_create(&thr[i], NULL, thr_func, &thr_data[i]))) {
            fprintf(stderr, "error: pthread_create, rc: %d\n", rc);
        }
    }
}
```

```

        return EXIT_FAILURE;
    }
}

for (i = 0; i < NUM_THREADS; ++i) {
    pthread_join(thr[i], NULL);
}

pthread_mutex_destroy(&count_mutex);
printf("Final counter value (WITH mutex): %d\n", shared_counter);
return EXIT_SUCCESS;
}
...

```

· To test without the mutex: Comment out the `pthread_mutex_lock` and `pthread_mutex_unlock` lines inside `thr_func`, recompile, and run several times. You will see inconsistent results.

—

Practical Session 2: Producer-Consumer Simulation

File: `practical2_producer_consumer.c`

```

``c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t mutex, empty, full;

void* producer(void* arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        int item = rand() % 100;
        sem_wait(&empty);
        sem_wait(&mutex);

```

```

    buffer[in] = item;
    printf("Producer produced item %d at index %d\n", item, in);
    in = (in + 1) % BUFFER_SIZE;

    sem_post(&mutex);
    sem_post(&full);
    sleep(1); // Simulate production time
}
pthread_exit(NULL);
}

void* consumer(void* arg) {
    for (int i = 0; i < NUM_ITEMS; i++) {
        sem_wait(&full);
        sem_wait(&mutex);

        int item = buffer[out];
        printf("Consumer consumed item %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty);
        sleep(2); // Simulate consumption time
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;

    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
}

```

```
    return 0;
}
...
```

- This uses three semaphores (mutex, empty, full) for synchronization.
- The sleep() calls simulate different speeds.

—

Practical Session 3: Semaphore Implementation in C

File: practical3_semaphore.c

```
```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_THREADS 5
#define MAX_CONCURRENT 3 // Counting semaphore allows 3 threads at a time

int shared_counter = 0;
sem_t counting_sem;

void* thread_func(void* arg) {
 int tid = *(int*)arg;
 sem_wait(&counting_sem); // Only MAX_CONCURRENT threads can pass

 printf("Thread %d is in the critical section.\n", tid);
 for (int i = 0; i < 1000; i++) {
 shared_counter++;
 }
 printf("Thread %d is leaving.\n", tid);

 sem_post(&counting_sem);
 pthread_exit(NULL);
}

int main() {
 pthread_t threads[NUM_THREADS];
 int tids[NUM_THREADS];

 sem_init(&counting_sem, 0, MAX_CONCURRENT);
```

```

for (int i = 0; i < NUM_THREADS; i++) {
 tids[i] = i;
 pthread_create(&threads[i], NULL, thread_func, &tids[i]);
}

for (int i = 0; i < NUM_THREADS; i++) {
 pthread_join(threads[i], NULL);
}

sem_destroy(&counting_sem);
printf("Final counter value: %d\n", shared_counter);
return 0;
}
...

```

· This demonstrates a counting semaphore, which allows a specified number of threads (e.g., 3) to access the resource concurrently.

—

## Practical Session 4: Shared Memory Programming

File: practical4\_shared\_memory.c

```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <unistd.h>
#include <semaphore.h>
#include <fcntl.h>

#define SHM_SIZE 1024
#define SEM_NAME "/my_semaphore"

int main() {
    int shmid;
    char *shared_memory;
    sem_t *sem;

    // Create shared memory

```

```

shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);
if (shmid == -1) {
    perror("shmget failed");
    exit(1);
}

// Attach shared memory
shared_memory = shmat(shmid, NULL, 0);
if (shared_memory == (char *) -1) {
    perror("shmat failed");
    exit(1);
}

// Create a named semaphore for synchronization
sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);
if (sem == SEM_FAILED) {
    perror("sem_open failed");
    exit(1);
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    exit(1);
}

if (pid == 0) { // Child process
    sem_wait(sem);
    strcpy(shared_memory, "Hello from Child!");
    sem_post(sem);
    shmdt(shared_memory);
    exit(0);
} else { // Parent process
    wait(NULL);
    sem_wait(sem);
    printf("Parent read: %s\n", shared_memory);
    sem_post(sem);
    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL);
    sem_close(sem);
    sem_unlink(SEM_NAME);
}

```

```
    return 0;  
}  
...
```

- This uses `shmget()` and `shmat()` for shared memory.
- A named POSIX semaphore synchronizes access between parent and child processes.
- Cleanup is done with `shmdt()` and `shmctl(IPC_RMID)`.

—